🏁 GitHub - Multiple view Camera calibration tool (ing...)

GitHub - cvlab-epfl/multiview_calib: Single and multiple view camera calibration tool

- 전체적으로 calibration 하는 방식은 동일할 것으로 보임

<details>
<summary>GitHub 글로된 부분 한국어로 정리</summary>

[GitHub에 나온 개요 참고]

이 도구는 시야가 겹치는 동기화된 여러 카메라의 **내부 파라미터(intrinsic)와 외부 파라미터(extrinsic)**를 계산할 수 있습니다.

- **내부 파라미터 추정**은 OpenCV의 카메라 보정 프레임워크를 기반으로 하며, 각 카메라에 대해 개별적으로 수행됩니다.
- **외부 파라미터 추정**에서는 먼저 선형 기법을 사용해 초기 해(외부 파라미터)를 구한 뒤, 번들 조정(bundle adjustment)을 통해 정밀하게 보정합니다.
- 최종 출력은 **카메라 포즈**로, 첫 번째 카메라 또는 전역 기준 좌표계에 대한 **내부 행렬, 왜곡 파라미터, 회전, 변환 값**을 포함합니다.

Intrinsic estimation (내부 파라미터 추정)

Checkerboard 이용해서 보정한다고 생각하면됨

1. 사용할 Checkerboard 준비
2. 다양한 각도와 거리에서 Checkboard 촬영
3. 프레임 추출 (촬영한 영상을 이미지 프레임으로 변환)
4. 보정 스크립트 실행

Extrinsics estimation (외부 파라미터 추정)

1. 카메라 동기화
2. 카메라 쌍별 상대 포즈 계산
3. 상대 포즈 연결
4. 번들 조정
5. 전역 좌표계로 변환

</details>

multiview_calib/docs/tutorial.pdf 참고해도 좋음

1 의존성 문제 해결

python 3.7.17에서 가능함 → pyenv 라는걸 사용했는데, 대충 여러 버전의 python을 설치할 수 있도록 하는 듯 (좀 더 찾아봐야되는데 모르겟당)

- 빌드 필수 패키지 설치

```
sudo apt update
sudo apt install -y make build-essential libssl-dev zlib1g-dev \
libbz2-dev libreadline-dev libsqlite3-dev wget curl llvm \
libncursesw5-dev xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev
liblzma-dev
```

- pyenv 설치

```
curl https://pyenv.run | bash
```

- 환경 변수 설정

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'command -v pyenv >/dev/null || export
PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
echo 'eval "$(pyenv virtualenv-init -)"' >> ~/.bashrc
```

- 환경 변수 설정 변경 내용 즉시 적용

```
source ~/.bashrc
```

- python 3.7.17 설치 + 가상환경 생성

```
pyenv install 3.7.17

# pyenv 가상환경 생성
pyenv virtualenv 3.7.17 calib_venv

# 활성화
pyenv activate calib_venv

# 기본 pip 업그레이드
python -m pip install -U pip setuptools wheel
```

- 관련 라이브러리 설치

```
pip install -r multiview_calib/requirements.txt
pip install -e .
```

Calibration Checkerboard Collection

- 사용한 checkerboard

2 Checkerboard (내부 파라미터 보정)

- 없다고 뜨면 설치하기

```
sudo apt update
sudo apt install ffmpeg
```

- checkerboard 영상 frame단위로 잘라서 .jpg로 변환하기

```
jimingbori@jm:~/proj/Test/multiview_calib$ ffmpeg -i
./Assets/vid_pattern.mp4 -r 0.5 frames/frame_%04d.jpg
```

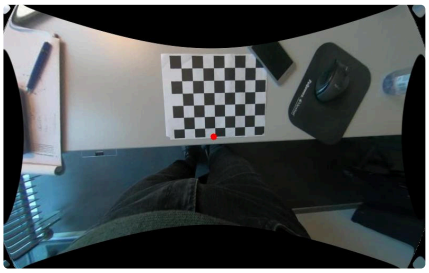
생각보다 checkerboard 영상 잘 해야됨 ! 싀앵 ... 이상하게 하면 넘어가지지도 않음 이럴수 ..

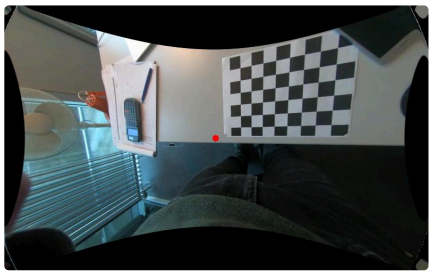
- 카메라 내부보정

```
jimingbori@jm:~/proj/Test/multiview_calib$ python -m
scripts.compute_intrinsics --folder_images ./frames -ich 6 -icw 8 -s 30
-t 24 -rm --debug
```

에러가 뜨지 않았다면 내부보정이 잘 됨

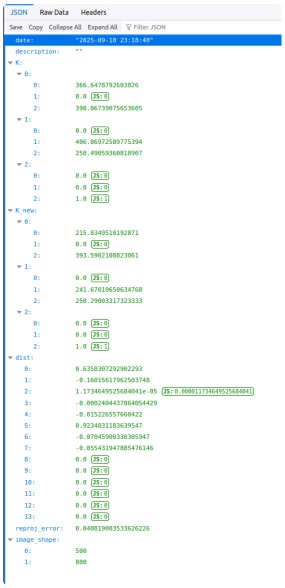
→ 결과물은 **./output/intrinsics.json** 으로 생성됨





/home/jimingbori/proj/Test/multiview_calib/output/undistorted

→ 빨간색 기준점이 찍히는걸 볼 수 있음



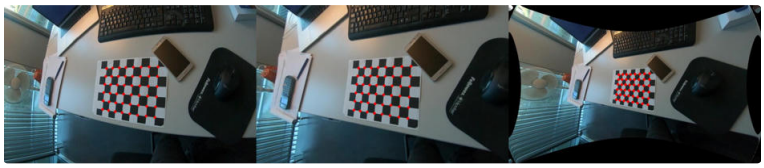
output/intrinsics.json

- **multiview_calib/output/jm.py 실행**

```python
import numpy as np
import cv2
import imageio

from multiview_calib import utils

inner_corners_height = 6
inner_corners_width = 8

intrinsics = utils.json_read("./output/intrinsics.json")
K = np.float32(intrinsics['K'])
K_new = np.float32(intrinsics['K_new'])
dist = np.float32(intrinsics['dist'])

img = imageio.v2.imread("./frames/frame_0027.jpg")
gray = img[:,:,0]

#-----------------
# get camera pose
#-----------------
ret, corners = cv2.findChessboardCorners(gray,
(inner_corners_height,inner_corners_width),
                                         cv2.CALIB_CB_ADAPTIVE_THRESH
+ cv2.CALIB_CB_NORMALIZE_IMAGE)

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
0.001)
image_points = cv2.cornerSubPix(gray, corners, (11,11), (-1,-1),
criteria)

object_points = np.zeros((inner_corners_height*inner_corners_width,3),
np.float32)
object_points[:,:2] =
np.mgrid[0:inner_corners_height,0:inner_corners_width].T.reshape(-1,2)

retval, rvec, tvec = cv2.solvePnP( object_points, image_points, K,
dist)

#-----------------
# undistort image
#-----------------
undist = cv2.undistort(img, K, dist, None, None)
undist_all = cv2.undistort(img, K, dist, None, K_new)

```

```
#-----------------
# project points
#-----------------
proj_obj = cv2.projectPoints(object_points, rvec, tvec, K, dist)[0].reshape(-1,2)
proj_obj_undist = cv2.projectPoints(object_points, rvec, tvec, K, None)[0].reshape(-1,2)
proj_obj_undist_all = cv2.projectPoints(object_points, rvec, tvec, K_new, None)[0].reshape(-1,2)

#-----------------
# visualisation
#-----------------

for (x,y) in proj_obj:
    img = cv2.circle(img, (int(x),int(y)), 4, [255,0,0], thickness=-1, lineType=-1)

for (x,y) in proj_obj_undist:
    undist = cv2.circle(undist, (int(x),int(y)), 4, [255,0,0], thickness=-1, lineType=-1)

for (x,y) in proj_obj_undist_all:
    undist_all = cv2.circle(undist_all, (int(x),int(y)), 4, [255,0,0], thickness=-1, lineType=-1)

imageio.imwrite("example.jpg", np.hstack([img, undist, undist_all]))
```

- 실행하면 **multiview_calib/example.jpg** 생성됨



(왼쪽) 원본 프레임 + (K, dist)로 재투영한 코너 점

(가운데) undistort(K, dist) + (K, no-dist)로 재투영

(오른쪽) undistort(K → K_new) + (K_new, no-dist)로 재투영

‼ 여기 내용은 더 찾아봐야 할듯 ‼

3 외부 파라미터 보정

- 따로 카메라가 없어서 제공된 example 이용해서 작성함
- **/home/jimingbori/proj/Test/multiview_calib/examples**

jmjm.py ← 코드 너무 길어유

```
#--------------------------------------------------------------------------
# Created By  : Leonardo Citraro leonardo.citraro@epfl.ch
# Date: 2020
# ---------------------------------------------------------------------------
import os
import numpy as np
import imageio
import itertools
import random
import logging
```

```
import cv2
import networkx as nx
from networkx.algorithms.tree.mst import maximum_spanning_edges

from multiview_calib import utils
from multiview_calib.singleview_geometry import undistort_points
from multiview_calib.twoview_geometry import
(compute_relative_pose, residual_error,
                               sampson_distance, draw_epilines,
triangulate, fundamental_from_relative_pose)
from multiview_calib.point_set_registration import
(estimate_scale_point_sets, procrustes_registration,
                                    point_set_registration,
apply_rigid_transform)
from multiview_calib.utils import colors as view_colors

logger = logging.getLogger(__name__)

def verify_view_tree(view_tree):
    G = nx.DiGraph()
    for view1, view2 in view_tree:
        G.add_edge(view1, view2)

    try:
        nx.algorithms.cycles.find_cycle(G)
        no_cycles_found = False
    except:
        no_cycles_found = True

    is_connected =
nx.number_connected_components(G.to_undirected())==1

    is_valid = no_cycles_found and is_connected

    return is_valid

def verify_landmarks(landmarks):
    for view,val in landmarks.items():
        if 'ids' not in val:
            return False, "landmarks file must contain the 'ids'"
        if 'landmarks' not in val:
            return False, "landmarks file must contain the
'landmarks'"
        unique = len(set(val['ids']))==len(val['ids'])
        if not unique:
            return False, "landmarks file contains duplicate IDs!"
        number = len(val['landmarks'])==len(val['ids'])
        if not number:
            return False, "the number of IDs and landmarks in
landmark file is not the same!"
        if len(val['landmarks'][0])!=2:
            return False, "the landmarks must be defined in a two-
dimensional space! {} wrong.".format(view)
    return True, ""

def _common_landmarks(landmarks1, landmarks2, ids1, ids2):

    ids_common = list(set(ids1).intersection(ids2))

    pts1 = np.vstack([landmarks1[ids1.index(i)] for i in
ids_common])
    pts2 = np.vstack([landmarks2[ids2.index(i)] for i in
ids_common])

    return pts1, pts2, ids_common

def visualise_epilines(view_tree, relative_poses, intrinsics,
landmarks, filenames,
                       output_path="output/relative_poses/"):

    for view1, view2 in view_tree:
```

```
        fnames = utils.json_read(filenames)

        img1 = imageio.imread(fnames[view1])
        img2 = imageio.imread(fnames[view2])

        K1 = np.float64(intrinsics[view1]['K'])
        K2 = np.float64(intrinsics[view2]['K'])
        dist1 = np.float64(intrinsics[view1]['dist'])
        dist2 = np.float64(intrinsics[view2]['dist'])

        img1_undist = cv2.undistort(img1.copy(), K1, dist1, None, K1)
        img2_undist = cv2.undistort(img2.copy(), K2, dist2, None, K2)

        pts1, pts2, _ = _common_landmarks(landmarks[view1]["landmarks"],
                                          landmarks[view2]["landmarks"],
                                          landmarks[view1]["ids"],
                                          landmarks[view2]["ids"])

        pts1_undist = undistort_points(pts1, K1, dist1)
        pts2_undist = undistort_points(pts2, K2, dist2)

        if 'F' in relative_poses[(view1, view2)]:
            F = np.array(relative_poses[(view1, view2)]['F'])
        else:
            Rd = np.array(relative_poses[(view1, view2)]['Rd'])
            td = np.array(relative_poses[(view1, view2)]['td'])

            F = fundamental_from_relative_pose(Rd, td, K1, K2)

        idx = np.arange(min(pts1_undist.shape[0],pts2_undist.shape[0]))
        np.random.shuffle(idx)
        img1_, img2_ = draw_epilines(img1_undist, img2_undist, pts1_undist[idx[:50]], pts2_undist[idx[:50]],
                                     F, None, linewidth=2, markersize=20)

        utils.mkdir(output_path)

        hmin = np.minimum(img1_.shape[0], img2_.shape[0])
        imageio.imsave(os.path.join(output_path, "{}_{}.jpg".format(view1, view2)), np.hstack([img1_[:hmin],img2_[:hmin]]))

def _print_relative_pose_info(F, Rd, td, pts1_undist, pts2_undist, verbose=2, print_prefix=''):

    if verbose>1:
        logging.info("{}\tFundamental matrix:\n{}\t\t{}\n{}\t\t{}\n{}\t\t{}".format(print_prefix,

print_prefix, F[0],

print_prefix, F[1],

print_prefix, F[2]))
        logging.info("{}\tRight camera position:\n{}\t\t{}".format(print_prefix,

print_prefix, utils.invert_Rt(Rd, td)[1].ravel()))
    if verbose>0:
        logging.info("{}\tResidual error: {}".format(print_prefix, residual_error(pts1_undist, pts2_undist, F)[0]))
        logging.info("{}\tSampson distance: {}".format(print_prefix, sampson_distance(pts1_undist, pts2_undist, F)[0]))

```

```
def compute_relative_poses(view_tree, intrinsics, landmarks,
                           method='8point', th=20, verbose=2,
print_prefix=''):

    relative_poses = {}
    for view1, view2 in view_tree:

        pts1, pts2, ids_common =
_common_landmarks(landmarks[view1]["landmarks"],

landmarks[view2]["landmarks"],

landmarks[view1]["ids"],

landmarks[view2]["ids"])

        K1 = np.float64(intrinsics[view1]['K'])
        K2 = np.float64(intrinsics[view2]['K'])
        dist1 = np.float64(intrinsics[view1]['dist'])
        dist2 = np.float64(intrinsics[view2]['dist'])

        Rd, td, F, pts1_undist, pts2_undist, tri, mask =
compute_relative_pose(pts1, pts2,

K1=K1, dist1=dist1,

K2=K2, dist2=dist2,

method=method, th=th)

        if verbose>0:
            logging.info("{}Computing relative pose of pair
{}:".format(print_prefix, [view1, view2]))
            logging.info("{}\t{} out of {} points considered
outliers.".format(print_prefix, sum(~mask), len(pts1_undist)))
        _print_relative_pose_info(F, Rd, td, pts1_undist[mask],
pts2_undist[mask], verbose, print_prefix)

        relative_poses[(view1, view2)] = {"F":F.tolist(),
"Rd":Rd.tolist(), "td":td.tolist(),

"triang_points":tri.tolist(),
                                          "ids":ids_common}

    return relative_poses

def visualise_cameras_and_triangulated_points(views, minimal_tree,
poses, triang_points,
                                              max_points=100,
path=None):
    import matplotlib.pyplot as plt
    from mpl_toolkits.mplot3d import Axes3D

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    all_points = []
    for i, (view1, view2) in enumerate(reversed(minimal_tree)):

        R1 = np.asarray(poses[view1]['R'], np.float64)
        t1 = np.asarray(poses[view1]['t'],
np.float64).reshape(3,1)
        R2 = np.asarray(poses[view2]['R'], np.float64)
        t2 = np.asarray(poses[view2]['t'],
np.float64).reshape(3,1)

        points_3d = np.asarray(triang_points[(view1, view2)]
['triang_points'], np.float64).T

        # pick some point at random
        idxs = np.arange(points_3d.shape[1])
```

```
        np.random.shuffle(idxs)
        idxs = idxs[:max_points]
        points_3d = points_3d[:,idxs]

        _, t1_inv = utils.invert_Rt(R1, t1)
        _, t2_inv = utils.invert_Rt(R2, t2)

        all_points.append(t1_inv.reshape(1,3))
        all_points.append(t2_inv.reshape(1,3))
        all_points.append(points_3d.T)

        color_view1 = view_colors[views.index(view1)]
        #color_view2 = view_colors[views.index(view2)]

        ax.scatter(points_3d[0], points_3d[1], points_3d[2],
c=np.array([color_view1]))

        p1 = t1_inv.ravel()
        p2 = t2_inv.ravel()
        ax.plot([p1[0],p2[0]], [p1[1],p2[1]], [p1[2],p2[2]],
'k',alpha=1, linewidth=1)
        ax.scatter(*p1, c=np.array([color_view1]), marker='s',
s=120, label=view1)
        ax.scatter(*p2, c=np.array([color_view1]), marker='x',
s=250)

    ax.set_xlabel('X Label')
    ax.set_ylabel('Y Label')
    ax.set_zlabel('Z Label')

    all_points = np.vstack(all_points)
    x_min, y_min, z_min = np.min(all_points, axis=0)
    x_max, y_max, z_max = np.max(all_points, axis=0)

    ax.set_xlim(x_min-0.1*np.abs(x_min), x_max+0.1*np.abs(x_max))
    ax.set_ylim(y_min-0.1*np.abs(y_min), y_max+0.1*np.abs(y_max))
    ax.set_zlim(z_min-0.1*np.abs(z_min), z_max+0.1*np.abs(z_max))

    plt.legend()
    plt.show()

    if path is not None:
        utils.mkdir(path)
        ax.view_init(15, 0)
        plt.savefig(path+"/cameras_points3d_1.jpg",
bbox_inches='tight')
        ax.view_init(15, 90)
        plt.savefig(path+"/cameras_points3d_2.jpg",
bbox_inches='tight')
        ax.view_init(15+90, 0)
        plt.savefig(path+"/cameras_points3d_3.jpg",
bbox_inches='tight')
        ax.view_init(15+90, 90)
        plt.savefig(path+"/cameras_points3d_4.jpg",
bbox_inches='tight')
        ax.view_init(20, -125)
        plt.savefig(path+"/cameras_points3d_5.jpg",
bbox_inches='tight')

def concatenate_relative_poses(minimal_tree, relative_poses,
method='cross-ratios', verbose=2, print_prefix=''):

    # initialize the graph with the first pair of view
    # The first camera will be the center of our coordinate system
for now
    pair0 = tuple(minimal_tree[0])
    poses = {pair0[0]: {"R":np.eye(3).tolist(),
"t":np.zeros((3,1)).tolist()},
             pair0[1]: {"R":relative_poses[pair0]['Rd'],
                        "t":relative_poses[pair0]['td']}}
```

```
    triang_points = {pair0: {"triang_points":relative_poses[pair0]['triang_points'],
                             "ids":relative_poses[pair0]['ids']}}

    def find_adjacent_pair(pair):
        adj_pair = None
        inverse = False
        for key,data in triang_points.items():
            if pair[0] in key :
                adj_pair = key
            elif pair[1] in key:
                adj_pair = key
                inverse = True
                break
        return adj_pair, inverse

    pairs = minimal_tree[1:]

    while len(pairs)>0:

        unmatched_pairs = []
        for curr_pair in pairs:
            curr_pair = tuple(curr_pair)

            adj_pair, inverse = find_adjacent_pair(curr_pair)

            if adj_pair is None:
                if curr_pair not in unmatched_pairs:
                    unmatched_pairs.append(curr_pair)
                continue

            if inverse:
                first_view, second_view = curr_pair[1], curr_pair[0]
            else:
                first_view, second_view = curr_pair

            # this is the new 0,0,0 point for the current pair
            R1 = np.asarray(poses[first_view]['R'], np.float32)
            t1 = np.asarray(poses[first_view]['t'], np.float32).reshape(3,1)

            # relative pose of the current pair
            Rd = np.asarray(relative_poses[curr_pair]['Rd'], np.float64)
            td = np.asarray(relative_poses[curr_pair]['td'], np.float64).reshape(3,1)
            if inverse:
                Rd, td = utils.invert_Rt(Rd, td)

            # triangulated points of the adjacent pair
            p3d_adj = np.float64(triang_points[adj_pair]['triang_points'])
            ids_adj = triang_points[adj_pair]['ids']

            # triangulated points of the current pair
            p3d = np.float64(relative_poses[curr_pair]['triang_points'])
            ids = relative_poses[curr_pair]['ids']

            # find common points
            p3d_adj_com, p3d_com, _ = _common_landmarks(p3d_adj, p3d, ids_adj, ids)

            # estimate scale between the two pairs
            if method=='cross-ratios':
                relative_scale, relative_scale_std = estimate_scale_point_sets(p3d_com, p3d_adj_com)
            elif method=='procrustes':
                relative_scale,_,_,_ = procrustes_registration(p3d_com, p3d_adj_com)
```

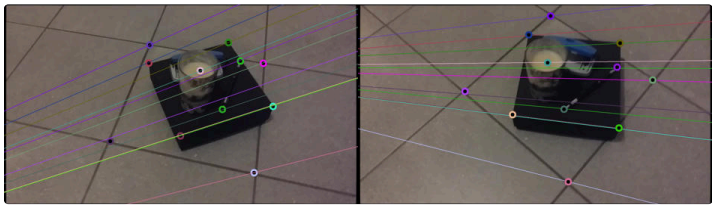
```
                _, relative_scale_std =
estimate_scale_point_sets(p3d_com, p3d_adj_com)
            else:
                raise ValueError("Unrecognized method
'{}'".format(method))

            # compute new camera pose for view2 of curr_pair
            R2 = np.dot(Rd, R1)
            t2 = np.dot(Rd, t1)+relative_scale*td

            if verbose>0:
                logging.info("{}Concatenating relative poses for
pair: {}".format(print_prefix, curr_pair))
                logging.info("{}\t Relative scale to {} n_points=
{}: {:0.3f}+-{:0.3f}".format(print_prefix, adj_pair, len(p3d_com),
relative_scale, relative_scale_std))
                logging.info("{}\t {} new position:
{}".format(print_prefix, second_view, utils.invert_Rt(R2, t2)
[1].ravel()))

            if relative_scale<0.1:
                logging.info("!!!! The relative scale for this
pair is quite low. Something may have gone wrong !!!!")

            # transform the triangulated points of the current
pair to the origin
            if inverse:
                R_inv, t_inv = utils.invert_Rt(R2, t2)
            else:
                R_inv, t_inv = utils.invert_Rt(R1, t1)

            p3d = np.dot(R_inv,
np.float64(p3d).T*relative_scale)+np.reshape(t_inv, (3,1))

            poses[second_view] = {'R':R2.tolist(),
't':t2.tolist()}
            triang_points[curr_pair] =
{'triang_points':p3d.T.tolist(),
                                          "ids":ids}

        if len(pairs)==len(unmatched_pairs):
            raise RuntimeError("The following pairs are not
connected to the rest of the network: {}".format(unmatched_pairs))
            break

        pairs = unmatched_pairs[:]

    return poses, triang_points

def build_view_graph(views, landmarks):

    G = nx.Graph()
    G.add_nodes_from(views)
    for view1, view2 in itertools.combinations(views, 2):

        pts1, pts2, _ = _common_landmarks(landmarks[view1]
["landmarks"],
                                          landmarks[view2]
["landmarks"],
                                          landmarks[view1]["ids"],
                                          landmarks[view2]["ids"])
        n_points = len(pts1)

        # checking if the pair of view has the minimum number
        # of points for computing the fundamental matrix
        if n_points>=8:
            G.add_edge(view1, view2, n_points=n_points)

    return G

def sample_random_view_tree(views, view0, landmarks):
```

```
350
351     G = build_view_graph(views, landmarks)
352
353     for s,t,data in G.edges(data=True):
354         data['weight'] = random.random()
355
356     T = nx.maximum_spanning_tree(G)
357     minimum_tree = list(nx.dfs_edges(T, source=view0))
358
359     return minimum_tree
360
361 def compute_relative_poses_robust(views, view_tree, intrinsics,
    landmarks,
362                                   method='8point', th=1,
    max_paths=5,
363                                   method_scale='cross-ratios',
    verbose=2):
364
365     G = build_view_graph(views, landmarks)
366
367     relative_poses = {}
368     for view1, view2 in view_tree:
369
370         if verbose>0:
371             logging.info("----------------------------------------
    ---------")
372             logging.info("Computing robust relative pose for pair
    {}->{}".format(view1, view2))
373             logging.info("Initial relative pose:")
374
375         view_tree = [[view1, view2]]
376         relative_pose = compute_relative_poses(view_tree,
    intrinsics, landmarks, method, th,
377                                                verbose,
    print_prefix='')
378
379         triangles = [ x for x in list(nx.all_simple_paths(G,
    view1, view2, 2)) if len(x)==3][:max_paths]
380         if len(triangles)==0:
381             logging.info("There are no other way to reach view {}
    from {}.".format(view2, view1))
382
383         if verbose>0:
384             logging.info("Number of additional paths found:
    {}".format(len(triangles)))
385
386         relative_pose_robust = {'Rd':[relative_pose[(view1,
    view2)]['Rd']],
387                                 'td':[relative_pose[(view1,
    view2)]['td']]}
388         for nodes in triangles:
389
390             view_tree = [nodes[:2],nodes[1:]]
391             _relative_pose = compute_relative_poses(view_tree,
    intrinsics, landmarks, method, th,
392                                                     verbose,
    print_prefix='\t')
393             pose, _ = concatenate_relative_poses(view_tree,
    _relative_pose, method_scale,
394                                                  verbose,
    print_prefix='\t')
395
396             relative_pose_robust['Rd'].append(pose[view2]['R'])
397             relative_pose_robust['td'].append(pose[view2]['t'])
398
399         Rd = np.mean(relative_pose_robust['Rd'], 0)
400         td = np.mean(relative_pose_robust['td'], 0)
401
402         pts1, pts2, ids_common =
    _common_landmarks(landmarks[view1]["landmarks"],
```

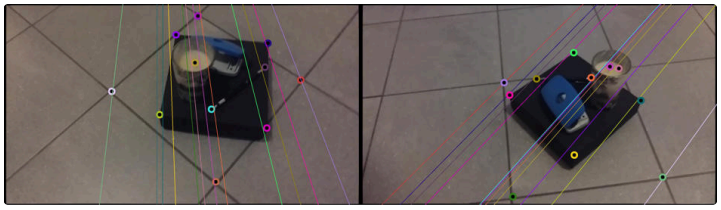
```
403 
    landmarks[view2]["landmarks"],
404 
    landmarks[view1]["ids"],
405 
    landmarks[view2]["ids"])
406 
407         K1 = np.float64(intrinsics[view1]['K'])
408         K2 = np.float64(intrinsics[view2]['K'])
409         dist1 = np.float64(intrinsics[view1]['dist'])
410         dist2 = np.float64(intrinsics[view2]['dist'])
411 
412         pts1_undist = undistort_points(pts1, K1, dist1)
413         pts2_undist = undistort_points(pts2, K2, dist2)
414 
415         F = fundamental_from_relative_pose(Rd, td, K1, K2)
416 
417         tri = triangulate(pts1_undist, pts2_undist,
418                           K1=K1, R1=np.eye(3), t1=np.zeros(3),
    dist1=None,
419                           K2=K2, R2=Rd, t2=td, dist2=None)
420 
421         relative_poses[(view1, view2)] = {'F':F.tolist(),
422                                           'Rd':Rd.tolist(),
423                                           'td':td.tolist(),
424 
    'triang_points':tri.tolist(),
425                                           'ids':ids_common}
426         if verbose>0:
427             logging.info("Final relative pose:")
428         _print_relative_pose_info(F, Rd, td, pts1_undist,
    pts2_undist, verbose, '')
429 
430     return relative_poses
431 
432 def global_registration(ba_poses, ba_points, landmarks_global):
433     """
434     Finds the rigid transformation between the two points clouds
    (B.A. points -> global landmarks).
435 
436     The transformation comprises a rotation, a translation and a
    scale factor.
437 
438     It is important to note that this transformation won't change
    the relative
439     position nor orientation of the cameras.
440     """
441 
442     src, dst, ids_common =
    _common_landmarks(ba_points['points_3d'],
443 
    landmarks_global['landmarks_global'],
444                                               ba_points['ids'],
445 
    landmarks_global['ids'])
446 
447     logging.info("src points: bundle adjustment optimized 3d
    points")
448     logging.info("dst points: global coordinates")
449 
450     scale, R, t, mean_dist = point_set_registration(src, dst,
    verbose=True)
451 
452     global_triang_points =
    {'points_3d':apply_rigid_transform(np.array(ba_points['points_3d']
    ),
453                                                                R,
    t, scale).tolist(),
454                                'ids': ba_points['ids']}
455 
456     R_inv, t_inv = utils.invert_Rt(R, t)
```

```
457
458     global_poses = {}
459     for cam,data in ba_poses.items():
460
461         R = np.array(data['R'])
462         t = np.reshape(data['t'], (3,1))
463
464         R2 = np.dot(R, R_inv)
465         t2 = np.dot(R, t_inv.reshape(3,-1)) +
t.reshape(3,-1)*scale
466
467         global_poses[cam] = {'K':data['K'], 'dist':data['dist'],
468                              'R':R2.tolist(),
't':t2.ravel().tolist()}
469
470     return global_poses, global_triang_points
471
472 def visualise_global_registration(global_poses, landmarks_global,
ba_poses, ba_points,
473                                   filenames,
output_path="output/global_registration"):
474     import matplotlib.pyplot as plt
475
476     utils.mkdir(output_path)
477
478     def plot(points3d, calib):
479         rvec = cv2.Rodrigues(np.array(calib['R']))[0]
480         tvec = np.array(calib['t'])
481         K = np.array(calib['K'])
482         dist = np.array(calib['dist'])
483
484         return cv2.projectPoints(points3d, rvec, tvec, K, dist)
[0].reshape(-1,2)
485
486     for view, calib in global_poses.items():
487
488         img = imageio.imread(filenames[view].format(view))
489
490         proj_glob =
plot(np.array(landmarks_global['landmarks_global']), calib)
491         proj_ba = plot(np.array(ba_points['points_3d']),
ba_poses[view])
492
493         plt.figure(figsize=(14,8))
494         plt.plot(proj_ba[:,0], proj_ba[:,1], 'r.', label='B.A
points')
495         plt.plot(proj_glob[:,0], proj_glob[:,1], 'b.',
label='Global points')
496         plt.imshow(img)
497         plt.show()
498         plt.legend()
499
plt.savefig(os.path.join(output_path,"global_registration_{}.jpg".
format(view)), bbox_inches='tight')
500
501 # 여기부터는 그냥 추가한거 ========================
502 if __name__ == "__main__":
503     logging.basicConfig(level=logging.INFO, format="%(message)s")
504
505     # 1) 예제 입력 로드 (경로는 simple_box 예시 기준)
506     setup_path       = "examples/simple_box/setup.json"         #
views, minimal_tree 포함
507     intrinsics_path  = "examples/simple_box/intrinsics.json"    #
{view: {K, dist}}
508     landmarks_path   = "examples/simple_box/landmarks.json"     #
{view: {landmarks, ids}}
509     filenames_path   = "examples/simple_box/filenames.json"     #
{view: "path/to/image.jpg"}
510
511     setup      = utils.json_read(setup_path)
512     intrinsics = utils.json_read(intrinsics_path)
```

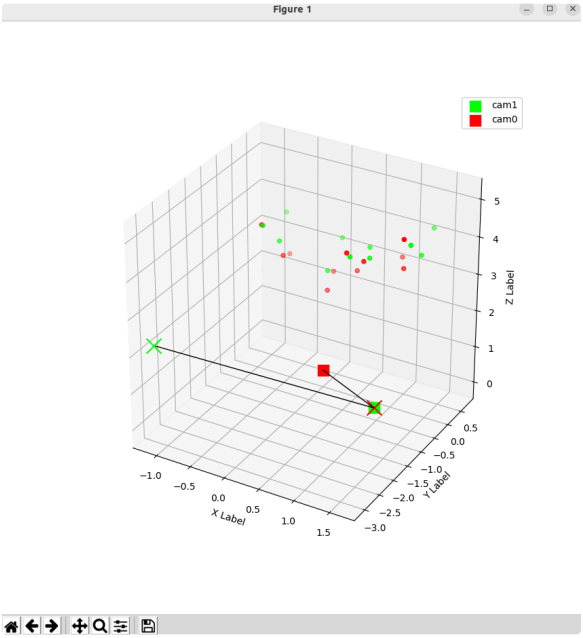
```
    landmarks  = utils.json_read(landmarks_path)

    views        = setup["views"]
    minimal_tree = setup["minimal_tree"]

    # (선택) 검증
    ok_tree = verify_view_tree(minimal_tree)
    ok_lm, msg = verify_landmarks(landmarks)
    if not ok_tree: raise RuntimeError("invalid minimal_tree")
    if not ok_lm:   raise RuntimeError(f"invalid landmarks:
{msg}")

    # 2) 쌍별 상대자세 + 삼각측량
    relative_poses = compute_relative_poses(
        view_tree=minimal_tree,
        intrinsics=intrinsics,
        landmarks=landmarks,
        method="8point",
        th=20,
        verbose=2
    )

    # 3) 전역 좌표계로 연결(스케일 포함)
    poses, triang_points = concatenate_relative_poses(
        minimal_tree=minimal_tree,
        relative_poses=relative_poses,
        method="cross-ratios",
        verbose=2
    )

    # 4) 시각화 결과 저장
    visualise_epilines(
        view_tree=minimal_tree,
        relative_poses=relative_poses,
        intrinsics=intrinsics,
        landmarks=landmarks,
        filenames=filenames_path,
        output_path="output/relative_poses/"
    )

    visualise_cameras_and_triangulated_points(
        views=views,
        minimal_tree=minimal_tree,
        poses=poses,
        triang_points=triang_points,
        max_points=200,
        path="output/cameras_3d"
    )

    print("DONE. Check:")
    print(" - output/relative_poses/*.jpg")
    print(" - output/cameras_3d/cameras_points3d_*.jpg")


```



camera0 , camera 1



camera 1, camera 2



plot 형태는 변경가능함...

각종 에러 문제 해결

- No module named ‘multiview_calib’

```
jimingbori@jm:~/proj/Test/multiview_calib$
/home/jimingbori/.pyenv/versions/3.7.17/envs/calib_venv/bin/python
/home/jimingbori/proj/Test/multiview_calib/output/jm.py
Traceback (most recent call last):
  File "/home/jimingbori/proj/Test/multiview_calib/output/jm.py", line
5, in <module>
    from multiview_calib import utils
ModuleNotFoundError: No module named 'multiview_calib'

jimingbori@jm:~/proj/Test/multiview_calib$ pip install -e .
```